

Customer Churn Prediction – Project Documentation

1. Overview

This project builds a machine learning model to predict whether a customer will **churn** (leave the service) based on their account information.

We used a real-world telecom dataset and trained several classifiers (Logistic Regression, Decision Tree, Random Forest, SVM) to find the best model.

The final model helps businesses identify at-risk customers and take action to retain them.

2. Dataset

- **Source:** IBM Telco Customer Churn dataset (available on GitHub/Kaggle)
 - **Size:** 7043 rows, 21 columns
 - **Target variable:** Churn (Yes = customer left, No = customer stayed)
 - **Features:** Customer demographics, account information, services subscribed, contract type, payment method, monthly charges, tenure, etc.
-

3. Project Structure

text

churn_prediction/

|

├─ churn_prediction.py # Main Python script

├─ churn_model.pkl # Saved trained model (Random Forest)

├─ scaler.pkl # Saved scaler for preprocessing

└─ README.md # This documentation

4. Requirements / Installation

Make sure you have **Python 3.7+** installed. Then install the required libraries:

bash

pip install pandas numpy matplotlib seaborn scikit-learn imbalanced-learn joblib

It's recommended to use a **virtual environment**:

bash

python -m venv venv

Activate it:

Windows: venv\Scripts\activate

Mac/Linux: source venv/bin/activate

5. How to Run

1. **Download or copy** the churn_prediction.py script.
2. Open a terminal in the project folder.
3. Activate your virtual environment (if any).
4. Run the script:

bash

python churn_prediction.py

5. Several plots will appear – close each one to let the script continue.
 6. After execution, you'll see model performance printed in the terminal, and the trained model + scaler will be saved.
-

6. Methodology – Step by Step

6.1 Data Cleaning

- Loaded the CSV file from GitHub.
- Converted TotalCharges to numeric (it had some empty strings).
- Dropped the 11 rows with missing TotalCharges.

- Removed the customerID column (not useful for prediction).

6.2 Exploratory Data Analysis (EDA)

We created several visualizations to understand the data:

- **Churn distribution** – showed that about 27% of customers churn (imbalanced).
- **Categorical features vs churn** – bar charts showing churn rate for each category (e.g., contract type, payment method).
- **Numerical features vs churn** – box plots for tenure, monthly charges, total charges.
- **Correlation heatmap** – to see relationships between numerical variables.

These plots help identify which factors might influence churn.

6.3 Feature Engineering & Preprocessing

- **Label Encoding** – converted binary Yes/No columns (like Partner, Dependents, Churn) to 0/1.
- **One-Hot Encoding** – for multi-category columns (like InternetService, Contract) to avoid ordinal assumptions.
- **Feature Scaling** – standardized numerical columns (tenure, MonthlyCharges, TotalCharges) so they have mean=0, variance=1. This is important for models like SVM and logistic regression.

After preprocessing, we had **31 features** (original categoricals expanded).

6.4 Handling Class Imbalance

The dataset had more non-churn customers ($\approx 73\%$) than churners ($\approx 27\%$).

To prevent the model from being biased toward the majority class, we used **SMOTE** (Synthetic Minority Oversampling Technique) on the **training data only**. This creates synthetic examples of the minority class (churners) to balance the classes.

6.5 Train-Test Split

- Split data: 80% training, 20% testing (stratified to keep same churn ratio).
- Training set size after SMOTE: 4130 samples per class.

6.6 Model Building

We trained four classifiers:

- Logistic Regression
- Decision Tree
- Random Forest (100 trees)
- Support Vector Machine (SVM)

All models were trained on the **resampled training data** and evaluated on the **original test data**.

6.7 Evaluation Metrics

For each model we calculated:

- **Accuracy** – overall correct predictions.
- **Precision** – of customers predicted to churn, how many actually churned.
- **Recall** – of actual churners, how many were correctly identified.
- **F1-Score** – harmonic mean of precision and recall (good for imbalanced data).
- **Confusion Matrix** – shows true/false positives/negatives.

We also printed a full classification report.

7. Results

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.73	0.49	0.70	0.58
Decision Tree	0.73	0.49	0.62	0.55
Random Forest	0.76	0.55	0.64	0.59
SVM	0.74	0.51	0.73	0.60

- **Random Forest** had the highest accuracy (76%) and best F1-score (0.59) – a good balance.
- **SVM** achieved the highest recall (73%), meaning it catches more actual churners, but at the cost of lower precision.
- For business use, you might choose the model that best matches your goal: if you want to **identify as many potential churners as possible** (even with some false alarms), SVM might be better. If you want **overall reliable predictions**, Random Forest is a solid choice.

Top 5 Important Features (from Random Forest):

1. tenure – how long the customer has stayed
2. TotalCharges – total amount billed
3. MonthlyCharges – monthly bill amount
4. PaymentMethod_Electronic check – using electronic check
5. Contract_Two year – having a two-year contract (negative influence – long contracts reduce churn)

These insights can guide retention strategies: focus on new customers, those with high monthly bills, and encourage longer contracts or automatic payments.

8. Saving and Loading the Model

The best model (Random Forest) and the scaler are saved using joblib:

```
python
```

```
import joblib
```

```
model = joblib.load('churn_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')
```

To make predictions on new customers, you must preprocess the new data **exactly** as done during training (same encoding, scaling using the loaded scaler). Example:

```
python
```

```
# new_customer = ... (DataFrame with same columns as original)
```

```
# Apply same encoding steps...
```

```
# new_customer_scaled =
```

```
scaler.transform(new_customer[['tenure', 'MonthlyCharges', 'TotalCharges']])
```

```
# prediction = model.predict(new_customer_processed)
```

```
# probability = model.predict_proba(new_customer_processed)[:,1] # churn probability
```

9. Future Improvements

- **Hyperparameter tuning** – use GridSearchCV to find optimal parameters for each model.
- **Cross-validation** – more robust performance estimation.
- **Try advanced algorithms** – XGBoost, LightGBM, Neural Networks.
- **Feature engineering** – create interaction terms or new features (e.g., average monthly charge = TotalCharges/tenure).
- **Model interpretability** – use SHAP or LIME to explain individual predictions.
- **Deploy as a web app** – build a simple interface with Streamlit or Flask.

10. Conclusion

This project successfully demonstrates an end-to-end machine learning pipeline for customer churn prediction. The Random Forest model achieved 76% accuracy and identified key churn drivers. The saved model can be integrated into business systems to proactively retain customers.

Author: Arun Kumar

Date: February 2026

Contact: arunkumarjamuaon@gmail.com